

Informal Description of the vib_nn

Introduction

This model is a neural network trained with a Variational Information Bottleneck (VIB). The key idea is: Instead of letting the network freely memorize information from $\mathbf{X}$ and possibly overfit, we explicitly control how much information the latent representation $\mathbf{z}$ is allowed to carry. So just like in the information bottleneck paper, we are trying to maximize predictive power by also compressing as much as we can.

1) Experimental setup

I run the whole training/evaluation pipeline for **latent dimensions 1 through 10** (and across multiple train/val/test splits). Thanks to that, I can see more clearly how performance and compression change when the bottleneck capacity changes.

2) Encoder outputs: (mu, logvar) instead of variance

The encoder does not output a deterministic latent vector. It outputs parameters of a diagonal Gaussian:

$$q(z|X) = \mathcal{N}(\mu(X), \text{diag}(\exp(\logvar(X))))$$

So it predicts **mu(X)** and **logvar(X)** (log-variance), not variance directly.

Why logvar? - Variance must be **positive**, but log-variance can be any real number. - Then **var = exp(logvar)** is automatically positive. - Learning logvar is usually numerically/optimisation wise more stable than learning var.

3) Guardrails (stability tricks)

To avoid absurd variances, I clamp logvar:

$$\logvar \leftarrow \text{clamp}(\logvar, -10, 5)$$

This prevents: - logvar very negative meaning var approx. 0 thus almost deterministic z (can cause collapse / brittle gradients) - logvar very positive meaning huge var thus super noisy z (training blows up / NaNs)

4) Reparameterization trick (so we can backprop through sampling)

To sample from $q(z|X)$ while keeping gradients, we do:

- $\varepsilon \sim \mathcal{N}(0, I)$
- $\text{std} = \exp(0.5 \cdot \logvar)$
- $z = \mu + \text{std} \odot \varepsilon$

Then **all prediction heads** take the same sampled $\mathbf{z}$ and produce predictions (Y, W>0, W, S=+1, S=-1).

5) Objective function: performance + bottleneck

The loss always has two parts:

$$L(\theta) = L_{\text{task}}(\theta) + \beta KL(\theta)$$

- L_{task} : the usual supervised multi task loss (classification/regression heads)
- KL : the bottleneck penalty

$$KL = KL(q(z|X) \parallel \mathcal{N}(0, I))$$

Intuition: **KL is basically “how many nats of info z is carrying about X .”** Higher KL = less compression meaning more info passed through.

6) Phase 1 (warmup, first 60 epochs): mostly just learn to predict

During warmup, β is tiny (almost no bottleneck pressure), so the objective is basically:

$$L \approx L_{\text{task}} + (\text{tiny}) \cdot KL$$

This lets the network learn good predictions first, while we observe what KL values it naturally settles into near an “unconstrained” solution. Near the end of warmup I store KL values to estimate a optimal budget.

7) Budget estimation

After warmup I set an information budget C based on the warmup KL:

$$C = \max(C_{\min}, m \cdot \text{median}(KL_{\text{warmup}}))$$

with $m < 1$, meaning: “after warmup, compress to around m times the warmup information.”

8) Phase 2 (dual ascent): adapt beta to enforce the budget

After warmup the loss is still:

$$L(\theta) = L_{\text{task}}(\theta) + \beta KL(\theta)$$

But now **beta is no longer fixed**. It gets updated like a Lagrange multiplier:

$$\beta \leftarrow \beta + \eta (KL_{\text{used}} - C)$$

Interpretation: - If $KL_{\text{used}} > C$: we’re passing **too much** information meaning increase β thus squeeze harder. - If $KL_{\text{used}} < C$: we’re passing **too little** information meaning decrease β thus relax the squeeze.

So beta is constantly calibrated to keep **KL** $\approx$ C .

9) What's being optimized each step?

Every gradient step updates the neural net parameters to reduce **both**: - the **performance loss** L_{task} (better predictions) - the **bottleneck term** via $\beta \cdot KL$ (compression control)

The task loss tries to encode whatever helps prediction; the KL pressure tries to remove anything unnecessary. The result is a representation that tends to keep only the most predictive information.
